

Prezentarea kit-ului de dezvoltare C8051F040DK

Comanda unui LED

1 Scopul lucrarii

Studentul ar trebui ca, la sfarsitul sedintei de laborator, sa aiba cunostintele de baza despre arhitectura nucleului CIP-51, chip-ul C8051F040, despre mediul hardware si software pentru dezvoltarea de aplicatii. Studentul se va familiariza cu toate acestea, realizand comanda unui LED.

2 Introducere teoretica

Kit-ul de dezvoltare C8051F040DK contine urmatoarele:

- 2.1. Hardware pentru dezvoltare: placa suport si placa de dezvoltare,
- 2.2. Software pentru dezvoltare: IDE (Integrated Development Environment).

2.1 Hardware pentru dezvoltare

2.1.1 Placa suport

Pe placa suport se afla conectori pentru modulul de alimentare, placa de dezvoltare si alte module specializate. Alimentarea modulelor specializate se realizeaza prin traseele magistralei de 9V (stanga), respectiv de 5V (dreapta). Alimentarea placii de dezvoltare se realizeaza printr-un cablu separat.

2.1.2 Modulul de alimentare

Modulul de alimentare se conecteaza la retea printr-un adaptor de 9V si distribuie tensiunea astfel:

- 9V catre placa de dezvoltare printr-un cablu,
- 9V catre prima linie de alimentare (stanga),
- 5V catre a doua linie de alimentare (dreapta).

2.1.2.1 Switch-uri si LED-uri

- switch-ul **SW1** conecteaza/deconecteaza linia de alimentare de 9V.
- switch-ul **SW2** conecteaza/deconecteaza linia de alimentare de 5V.
- led-ul **Led1** indica alimentarea magistralei de 9V.
- led-ul **Led2** indica alimentarea magistralei de 5V.

2.1.2.2 Conectori de alimentare

- conectorul **J3** – intrarea de la adaptorul de retea
- conectorul **J4** – iesire de 9V catre placa de dezvoltare
- conectorul **9V** – alimenteaza magistrala de 9V
- conectorul **5V** – alimenteaza magistrala de 5V
- conectorii **GND** – masa magistralelor de 9V, respectiv 5V.

2.1.3 Placa de dezvoltare

Chip-ul C8051F040 se afla pe o placa pentru evaluare si dezvoltare de software, care are numeroase conexiuni de intrare/iesire pentru a facilita crearea de prototipuri. A se vedea Fig. 2.1.3. pentru locatiile acestor conexiuni de intrare/iesire.

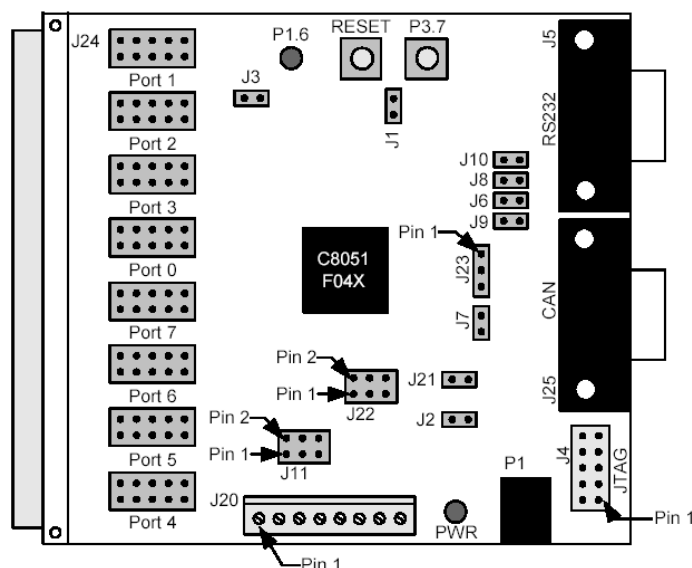


Fig. 2.1.3. Hardware pentru dezvoltare

In continuare, vom prezenta componentele hardware.

2.1.3.1 Chip

Chip-ul C8051F040 va fi prezentat separat la capitolul 2.1.4.

2.1.3.2 Switch-uri si LED-uri

- switch-ul **S1**, etichetat **RESET** este conectat la pinul **RESET** al chip-ului. Daca se apasa S1, sistemul va fi in stare de reset-hardware.
- switch-ul **S2**, etichetat **P3.7**, este conectat la pinul 7 al portului P3. Apasarea switch-ului S2 conecteaza pinul portului la masa. Scoaterea jumperului J1 are ca efect deconectarea lui S2 de la pinii porturilor.
- led-ul **D2**, etichetat **PWR**, indica alimentarea placii de dezvoltare la 9V.
- led-ul **D3**, etichetat **P1.6**, este conectat la pinul 6 portului de uz general P1.

2.1.3.3 Interfata seriala (J5) si interfata CAN (Controller Area Network) (J25)

Circuitul de emisie-receptie RS232 si interfata CAN faciliteaza comunicatiile seriale.

2.1.3.4 Interfata JTAG (J4)

Conectorul JTAG (**J4**) da acces la pinii JTAG ai chip-ului C8051F040.

Este folosit pentru:

- conectarea la placa de dezvoltare a adaptorului USB,
- depanare,
- programarea memoriei Flash.

2.1.3.5 Intrari/Iesiri analogice (J11, J20)

Conectorul **J11** cupleaza iesirile convertoarelor digital analogice la portul de uz general **J24**. Blocul terminal **J20** este folosit, de regula, in lucrul cu tensiunea de referinta si cu semnalele analogice de intrare.

2.1.3.6 Oscilatorul extern

Placa are un cristal extern cu frecventa de rezonanta de 22.1184 Mhz.

2.1.3.7 Conectorii porturilor I/O (J12-J19)

Sunt opt porturi paralele cu cate zece pini fiecare.

2.1.3.8 Conectorul I/O (J24)

Are 96 de pini si este folosit pentru a conecta module anexe la placa principala. J24 da acces la multe dintre semnalele pinilor.

2.1.3.9 Conectorul VREF (J22)

VREF (Voltage Reference) ofera posibilitatea folosirii tensiunii de referinta interna a placii.

2.1.4 System-on-a-chip (SOC)

Dispozitivele C8051F040 sunt microcontrolere, complet integrate in sisteme monocip sau SOC (System-on-a-Chip) cu 64 de pini digitali I/O. Structura SOC-ului este prezentata in Fig. 2.1.4a si in diagrama din Fig. 2.1.4b.

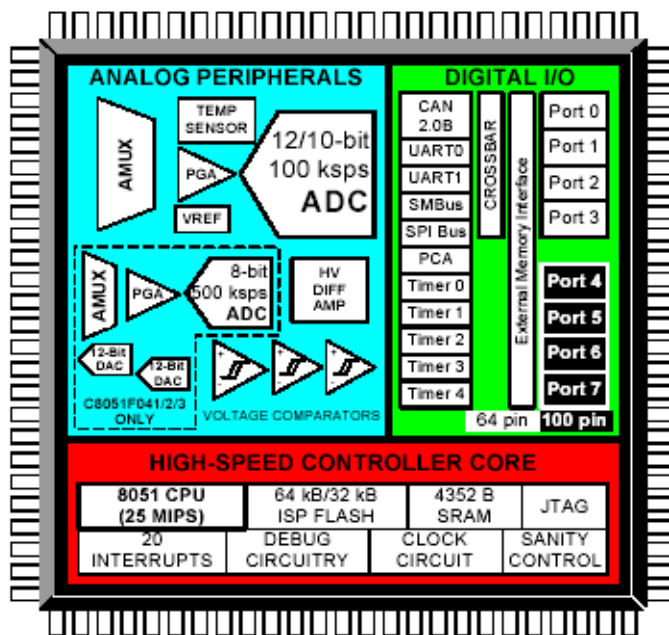


Fig. 2.1.4a Structura SOC-ului

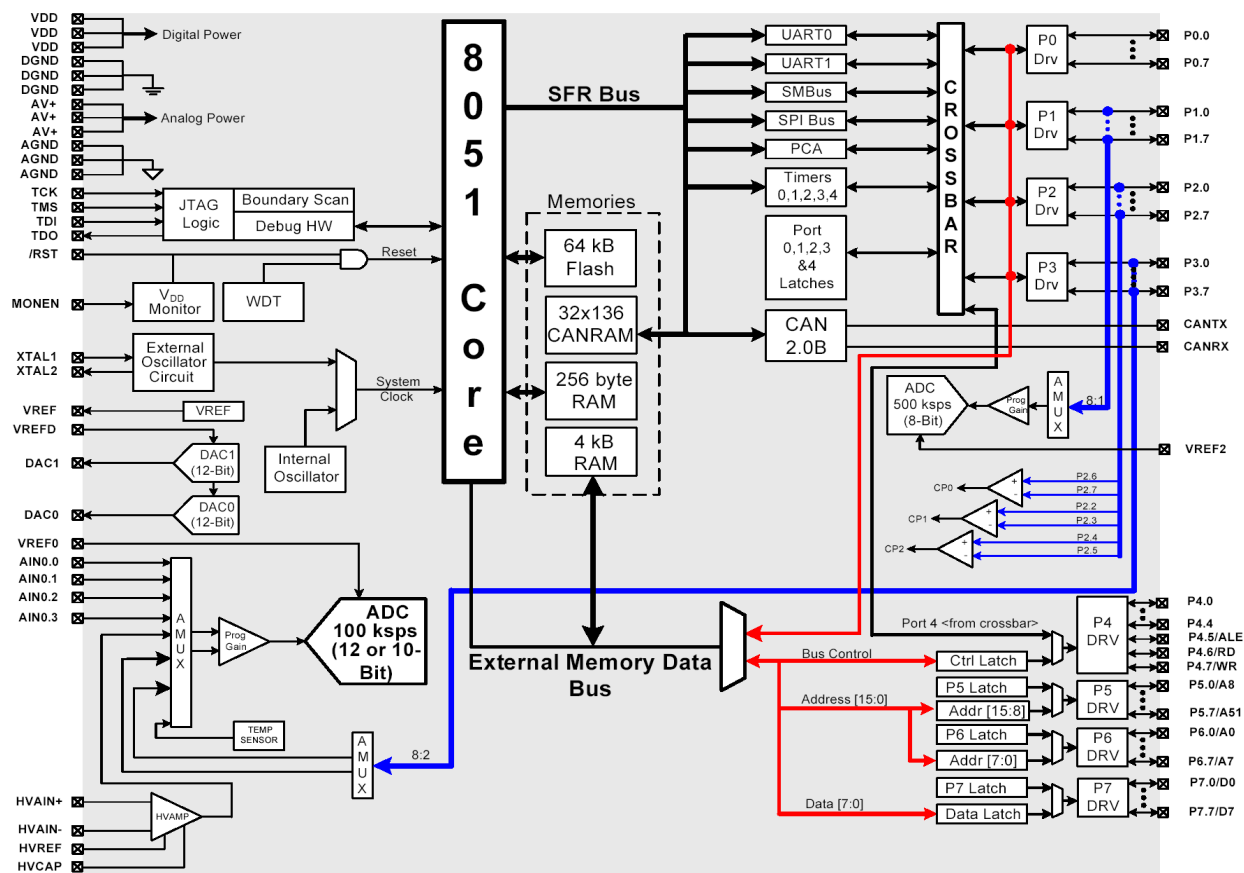


Fig. 2.1.4b Diagrama bloc a C8051F040

2.1.4.1 Nucleul 8051

2.1.4.1.1 Compatibilitatea cu nucleul microcontrolerului 8051.

SOC-ul C8051F040 foloseste ca nucleu CIP-51, proprietate a Silicon Labs, fiind compatibil cu arhitectura nucleului clasic de 8051.

2.1.4.1.2 Rezultate imbunatatite fata de 8051 clasic

CIP-51 are o arhitectura de tip pipeline care ii imbunatateste foarte mult performantele fata de arhitectura standard 8051. De exemplu, intr-un 8051 clasic, toate instructiunile, exceptand MUL si DIV au nevoie de 12 sau 24 stari pentru executie, cu tactul sistemului functionand de la 12 pana la 24 MHz. Nucleul CIP-51 executa 70% din instructiunile sale intr-unul sau 2 cicluri de ceas de sistem si are doar 4 instructiuni, care dureaza mai mult de 4 stari.

Daca CIP-51 functioneaza la 25 MHz, are un rezultat maxim de 25 MIPS. Figura 2.1.4.1.2 arata o comparatie intre rezultatele maxime ale mai multor nuclee de microcontrolere pe 8 biti.

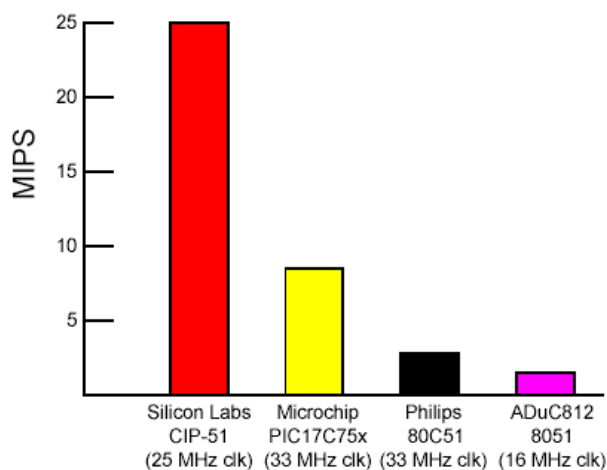


Fig. 2.1.4.1.2. Comparatie intre vitezele de executie maxime ale diferitelor tipuri de MCU (Microcontroler Unit)

2.1.4.1.3 Trasaturi suplimentare fata de 8051 clasic

C8051F040 include mai multe imbunatatiri importante in nucleul CIP-51 si in periferice pentru a mari performanta totala si pentru a facilita utilizarea lui in aplicatii.

Sunt 20 de surse de intrerupere in CIP-51, fata de 8051 standard care avea 7, permitand perifericelor digitale si analogice sa intrerupa controlerul. Un sistem care este programat prin intreruperi necesita o interventie mai mica din partea MCU (Microcontroler Unit), facandu-l mai eficient.

Sunt mai mult de 7 surse de resetare pentru MCU:

- Vdd monitor
- Watchdog Timer
- Detector de lipsa a ceasului
- Detector de nivel al tensiunii de la Comparator 0
- Reset software fortat
- Pinul de intrare CNVSTR0
- Pinul /RST

MCU are un generator intern de ceas (24.5MHz) care este folosit ca ceas de sistem dupa orice reset. Sursa de ceas poate fi comutata pe oscilatorul extern de frecventa mai mica. In Fig. 2.1.4.1.3 puteti observa schema sistemului de reset si a celui de ceas.

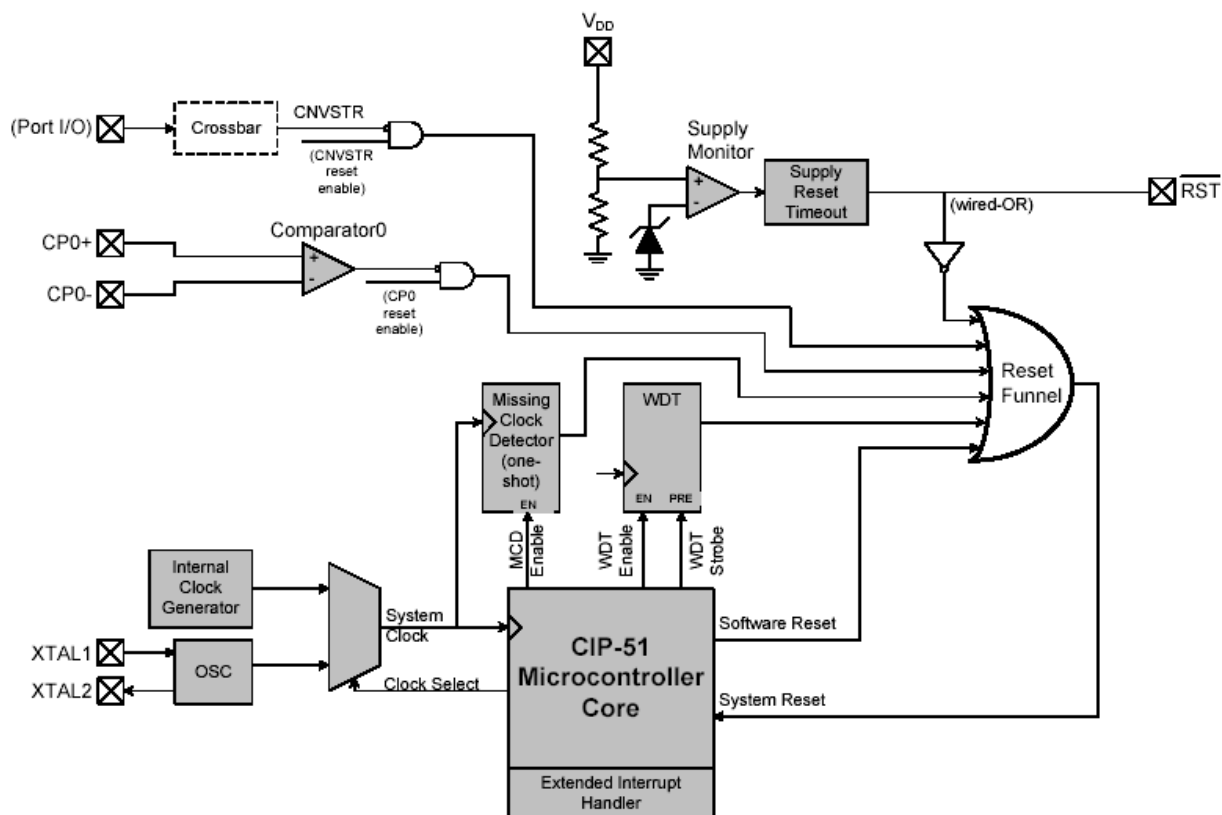


Figura 2.1.4.1.3. Ceasul si resetul din chip

2.1.4.1.4 Setul de instructiuni

Tabelul 2.1.4.1.4. Setul de instructiuni al nucleului CIP-51

Mnemonic	Description	Bytes	Clock Cycles
Arithmetic Operations			
ADD A, Rn	Add register to A	1	1
ADD A, direct	Add direct byte to A	2	2
ADD A, @Ri	Add indirect RAM to A	1	2
ADD A, #data	Add immediate to A	2	2
ADDC A, Rn	Add register to A with carry	1	1
ADDC A, direct	Add direct byte to A with carry	2	2
ADDC A, @Ri	Add indirect RAM to A with carry	1	2
ADDC A, #data	Add immediate to A with carry	2	2
SUBB A, Rn	Subtract register from A with borrow	1	1
SUBB A, direct	Subtract direct byte from A with borrow	2	2
SUBB A, @Ri	Subtract indirect RAM from A with borrow	1	2
SUBB A, #data	Subtract immediate from A with borrow	2	2
INC A	Increment A	1	1
INC Rn	Increment register	1	1
INC direct	Increment direct byte	2	2
INC @Ri	Increment indirect RAM	1	2
DEC A	Decrement A	1	1

Mnemonic	Description	Bytes	Clock Cycles
DEC Rn	Decrement register	1	1
DEC direct	Decrement direct byte	2	2
DEC @Ri	Decrement indirect RAM	1	2
INC DPTR	Increment Data Pointer	1	1
MUL AB	Multiply A and B	1	4
DIV AB	Divide A by B	1	8
DA A	Decimal adjust A	1	1
Logical Operations			
ANL A, Rn	AND Register to A	1	1
ANL A, direct	AND direct byte to A	2	2
ANL A, @Ri	AND indirect RAM to A	1	2
ANL A, #data	AND immediate to A	2	2
ANL direct, A	AND A to direct byte	2	2
ANL direct, #data	AND immediate to direct byte	3	3
ORL A, Rn	OR Register to A	1	1
ORL A, direct	OR direct byte to A	2	2
ORL A, @Ri	OR indirect RAM to A	1	2
ORL A, #data	OR immediate to A	2	2
ORL direct, A	OR A to direct byte	2	2
ORL direct, #data	OR immediate to direct byte	3	3
XRL A, Rn	Exclusive-OR Register to A	1	1
XRL A, direct	Exclusive-OR direct byte to A	2	2
XRL A, @Ri	Exclusive-OR indirect RAM to A	1	2
XRL A, #data	Exclusive-OR immediate to A	2	2
XRL direct, A	Exclusive-OR A to direct byte	2	2
XRL direct, #data	Exclusive-OR immediate to direct byte	3	3
CLR A	Clear A	1	1
CPL A	Complement A	1	1
RL A	Rotate A left	1	1
RLC A	Rotate A left through Carry	1	1
RR A	Rotate A right	1	1
RRC A	Rotate A right through Carry	1	1
SWAP A	Swap nibbles of A	1	1
Data Transfer			
MOV A, Rn	Move Register to A	1	1
MOV A, direct	Move direct byte to A	2	2
MOV A, @Ri	Move indirect RAM to A	1	2
MOV A, #data	Move immediate to A	2	2
MOV Rn, A	Move A to Register	1	1
MOV Rn, direct	Move direct byte to Register	2	2
MOV Rn, #data	Move immediate to Register	2	2
MOV direct, A	Move A to direct byte	2	2
MOV direct, Rn	Move Register to direct byte	2	2
MOV direct, direct	Move direct byte to direct byte	3	3
MOV direct, @Ri	Move indirect RAM to direct byte	2	2

Mnemonic	Description	Bytes	Clock Cycles
MOV direct, #data	Move immediate to direct byte	3	3
MOV @Ri, A	Move A to indirect RAM	1	2
MOV @Ri, direct	Move direct byte to indirect RAM	2	2
MOV @Ri, #data	Move immediate to indirect RAM	2	2
MOV DPTR, #data16	Load DPTR with 16-bit constant	3	3
MOVC A, @A+DPTR	Move code byte relative DPTR to A	1	3
MOVC A, @A+PC	Move code byte relative PC to A	1	3
MOVX A, @Ri	Move external data (8-bit address) to A	1	3
MOVX @Ri, A	Move A to external data (8-bit address)	1	3
MOVX A, @DPTR	Move external data (16-bit address) to A	1	3
MOVX @DPTR, A	Move A to external data (16-bit address)	1	3
PUSH direct	Push direct byte onto stack	2	2
POP direct	Pop direct byte from stack	2	2
XCH A, Rn	Exchange Register with A	1	1
XCH A, direct	Exchange direct byte with A	2	2
XCH A, @Ri	Exchange indirect RAM with A	1	2
XCHD A, @Ri	Exchange low nibble of indirect RAM with A	1	2
Boolean Manipulation			
CLR C	Clear Carry	1	1
CLR bit	Clear direct bit	2	2
SETB C	Set Carry	1	1
SETB bit	Set direct bit	2	2
CPL C	Complement Carry	1	1
CPL bit	Complement direct bit	2	2
ANL C, bit	AND direct bit to Carry	2	2
ANL C, /bit	AND complement of direct bit to Carry	2	2
ORL C, bit	OR direct bit to carry	2	2
ORL C, /bit	OR complement of direct bit to Carry	2	2
MOV C, bit	Move direct bit to Carry	2	2
MOV bit, C	Move Carry to direct bit	2	2
JC rel	Jump if Carry is set	2	2/3
JNC rel	Jump if Carry is not set	2	2/3
JB bit, rel	Jump if direct bit is set	3	3/4
JNB bit, rel	Jump if direct bit is not set	3	3/4
JBC bit, rel	Jump if direct bit is set and clear bit	3	3/4
Program Branching			
ACALL addr11	Absolute subroutine call	2	3
LCALL addr16	Long subroutine call	3	4
RET	Return from subroutine	1	5
RETI	Return from interrupt	1	5
AJMP addr11	Absolute jump	2	3
LJMP addr16	Long jump	3	4
SJMP rel	Short jump (relative address)	2	3
JMP @A+DPTR	Jump indirect relative to DPTR	1	3
JZ rel	Jump if A equals zero	2	2/3

Mnemonic	Description	Bytes	Clock Cycles
JNZ rel	Jump if A does not equal zero	2	2/3
CJNE A, direct, rel	Compare direct byte to A and jump if not equal	3	3/4
CJNE A, #data, rel	Compare immediate to A and jump if not equal	3	3/4
CJNE Rn, #data, rel	Compare immediate to Register and jump if not equal	3	3/4
CJNE @Ri, #data, rel	Compare immediate to indirect and jump if not equal	3	4/5
DJNZ Rn, rel	Decrement Register and jump if not zero	2	2/3
DJNZ direct, rel	Decrement direct byte and jump if not zero	3	3/4
NOP	No operation	1	1

Notes on Registers, Operands and Addressing Modes:
Rn - Register R0-R7 of the currently selected register bank. @Ri - Data RAM location addressed indirectly through R0 or R1. rel - 8-bit, signed (two's complement) offset relative to the first byte of the following instruction. Used by SJMP and all conditional jumps. direct - 8-bit internal data location's address. This could be a direct-access Data RAM location (0x00-0x7F) or an SFR (0x80-0xFF). #data - 8-bit constant #data16 - 16-bit constant bit - Direct-accessed bit in Data RAM or SFR addr11 - 11-bit destination address used by ACALL and AJMP. The destination must be within the same 2K-byte page of program memory as the first byte of the following instruction. addr16 - 16-bit destination address used by LCALL and LJMP. The destination may be anywhere within the 64K-byte program memory space. There is one unused opcode (0xA5) that performs the same function as NOP. All mnemonics copyrighted © Intel Corporation 1980.

2.1.4.2 Memoria din SOC

CIP-51 are o configuratie a adreselor de date si de program asemanatoare cu 8051 clasic. Sunt doua spatii separate de memorie: memoria de program si memoria de date. Memoria de program si memoria de date impart acelasi spatiu al adreselor, dar sunt accesate prin tipuri diferite de instructiuni.

2.1.4.2.1 Memoria de program

Memoria de program consta in 64 kB de memorie Flash reprogramabila organizata in blocuri continue de cate 512 octeti, cu adrese de la 0x0000 la 0xFFFF (vezi Fig. 2.1.4.2.1). Blocul de 512 octeti de la adresa 0xFE00 pana la 0xFFFF este rezervat de producator si nu este disponibil programatorului. Memoria de program este in mod normal read-only. Cu toate acestea, CIP-51 poate scrie in memoria de program prin setarea bitului Program Store Write Enable (PSCCTL.0 – bitul 0 al registrului Program Store Read/Write Control) si utilizarea instructiunii MOVX. Acesta facilitate ofera un mecanism de actualizare codului si de folosire a memoriei de program pentru stocarea non-volatila a datelor.

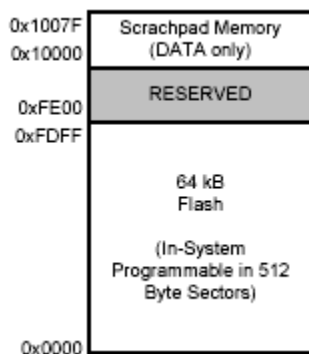


Fig. 2.1.4.2.1 Memoria de program (Flash)

2.1.4.2.2 Memoria de date

CIP-51 include 256 bytes de memorie de date RAM (vezi Fig. 2.1.4.2.2.). Primii 128 octeti din memoria de date sunt folositi pentru registrele generale si ca zona de scratch-pad. Pentru primii 128 de octeti se poate folosi atat adresarea directa, cat si adresarea indirecta. Locatiile de la 0x00 la 0xFF sunt adresate ca 4 bank-uri de registre generale, fiecare continand 8 registre de 8 biti. Urmatorii 16 octeti (aflati la adresele 0x20, pana la 0x2F), pot fi adresati fie ca octeti, fie ca 128 de biti adresabili individual, folosind adresarea directa.

Ceilalti 128 de octeti (cei superiori) pot fi accesati doar indirect. Aceasta zona ocupa acelasi spatiu de adrese ca si registrele speciale (SFR), dar este separata din punct de vedere fizic de acestea. Modul de adresare folosit de instructiuni pentru adresarea locatiilor mai mari ca 0x7F determina daca nucleul de procesare acceseaza cei 128 de octeti superiori din memoria de date sau registrele speciale. Instructiunile care folosesc adresarea directa vor accesa registrele speciale, in timp ce instructiunile care folosesc adresarea indirecta acceseaza cei 128 de octeti superiori din memoria de date.

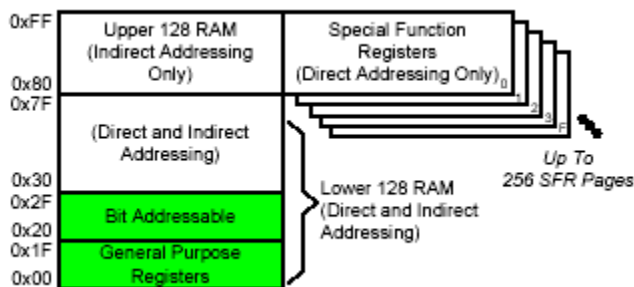


Fig.2.1.4.2.2. Memoria de date (RAM)

2.1.4.2.3 Interfata cu memoria externa

CIP-51 mai are in chip un bloc de 4 kB RAM si o interfata de memorie externa pentru a accesa memoria de date din afara chipului, respectiv perifericele mapate (vezi Fig. 2.1.4.2.3).

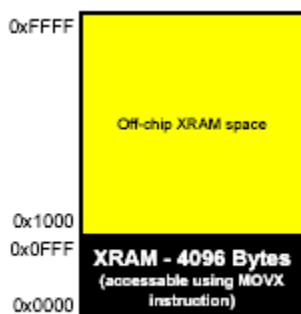


Fig. 2.1.4.2.3. Spatiul de adrese al memoriei externe

2.1.4.2.4 Registre de uz general

Locatiile de memorie de la 0x00 la 0x1F pot fi adresate ca patru bank-uri de registre de uz general. Fiecare bank e format din opt registre de un byte, etichetate R0 pana la R7. Numai unul dintre aceste bank-uri este activ la un moment dat. Bank-ul de registre activ este selectat prin intermediul bitilor RS0 (PSW.3) si RS1 (PSW.4) din registrul special Program Status Word (PSW). Aceasta faciliteaza schimbarea rapida a contextului la intrarea in subrutine sau rutine de deservire a intreruperilor. Registrele R0 si R1 sunt registre index pentru modurile de adresare indirecta.

2.1.4.2.5 Locatii adresabile pe bit

In plus fata de accesul direct la memoria organizata pe octeti, CIP-51 pune la dispozitie un spatiu de memorie de 16 octeti adresabil pe bit. Fiecare bit are o adresa intre 0x00 si 0x7F. Exemple:

- bitul 0 al octetului cu adresa 0x20 are adresa de bit 0x00
- bitul 7 al octetului cu adresa 0x20 are adresa de bit 0x07
- bitul 7 al octetului cu adresa 0x2F are adresa de bit 0x7F.

Diferenta intre accesarea memoriei pe bit sau pe octet este data de tipul instructiunii folosite (operanzi sursa si destinatie pe bit sau pe octet). Limbajul de asamblare MCS-51™ permite o notatie alternative pentru adresarea pe bit, de forma XX.B, unde XX este adresa octetului, iar B este pozitia bitului in interiorul octetului. De exemplu, instructiunea:

```
MOV    C, 22.3h
```

muta valoarea de bit de la adresa 0x13 (bitul 3 al octetului cu adresa 0x22) in flag-ul Carry.

2.1.4.2.6 Stiva

Stiva poate fi localizata oriunde in memoria de date. Zona de stiva este definita cu registrul special Stack Pointer (SP). Stack Pointer-ul va indica intotdeauna ultima locatie folosita. Urmatoarea valoare introdusa in stiva va fi plasata la locatia SP+1, apoi registrul SP este incrementat. La reset SP-ul este initializat cu valoarea 0x07, astfel ca prima valoare introdusa in stiva este plasata la locatia 0x08 (primul registru al bank-ului 1). Astfel, daca se doreste utilizarea a mai mult de un bank de registre, SP-ul ar trebui initializat cu o valoare mai mare care nu va fi folosita pentru stocare de date. Adancimea stivei poate ajunge la 256 de octeti.

2.1.4.2.7 Registre speciale

Locatiile de memorie direct accesabile de la 0x80 pana la 0xFF reprezinta registrele speciale. Acestea ofera controlul asupra schimbului de date dintre CIP-51 si periferice. CIP-51 contine atat registrele speciale din nucleul clasic 8051, cat si registre suplimentare folosite pentru configurarea si accesarea perifericelor suplimentare continute de acesta. Aceasta permite adaugarea de noi functionalitati pastrandu-se compatibilitatea cu setul clasic de instructiuni. Registrele speciale sunt accesate de fiecare data cand se foloseste adresarea directa a locatiilor de memorie de la 0x80 la 0xFF. Registrele a caror adresa se termina in 0x0 sau 0x8 (de exemplu P0, TCON, P1, SCON, IE) sunt adresabile pe bit, cat si pe octet. Toate celelalte registre sunt accesabile doar pe octet. Adresele neocupate sunt pastrate pentru implementare ulterioara. Accesarea acestor adrese nu are un efect bine determinat si trebuie evitata. Pentru o descriere detaliata a fiecarui registru, consultati documentatia microcontroler-ului (fisierul C8051F04xRev1_4.pdf, pag. 147).

Paginarea registrelor speciale

CIP-51 permite paginarea registrelor speciale, putand fi mapate mai multe registre in spatiul de adrese de la 0x80 la 0xFF. Memoria alocata registrelor speciale este impartita in 256 de pagini. In acest fel, fiecare adresa de la 0x80 la 0xFF poate accesa pana la 256 de registre speciale. Microcontroller-ul C8051F040 foloseste 5 pagini de registre speciale: 0, 1, 2, 3 si F. Paginile registrelor speciale sunt selectate cu ajutorul registrului de selectare a paginii (Special Function Register Page Selection – SFRPAGE). Pentru scrierea si citirea in/din registrele speciale se procedeaza astfel:

- se selecteaza pagina dorita cu ajutorul registrului SFRPAGE
- se foloseste adresarea directa pentru a scrie sau citi in/din registrul special (instructiunea MOV).

2.1.5 Adaptorul USB

Adaptorul USB este interfata dintre portul USB al PC-ului si circuitul pentru depanare/programare in sistem al SOC-ului.

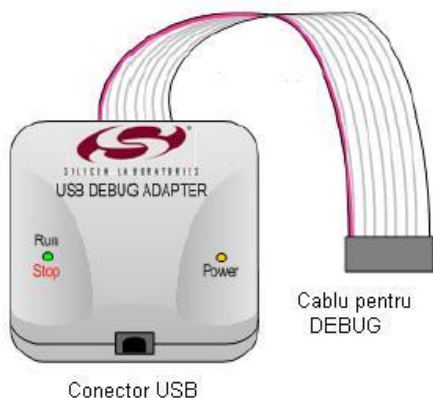


Fig. 2.1.5. Adaptorul USB

2.1.6 Instalare hardware

Placa de dezvoltare se conecteaza la un computer care ruleaza IDE prin adaptorul USB ca in Fig. 2.1.4. Pasii sunt urmatoarii:

- Conecteaza adaptorul USB pentru depanare la conectorul JTAG de pe placa de dezvoltare prin magistrala cu 10 pini.
- Conecteaza unul din capetele cablului USB la conectorul USB al adaptorului USB.
- Conecteaza celalalt capat al cablului la un port USB al computerului.
- Conecteaza adaptorul de retea la mufa **J3** a modulului de alimentare.

Nota:

- Decuplati alimentarea inainte de a conecta sau deconecta magistrala de la interfata JTAG. Conectarea si deconectarea magistralei, cand placa este alimentata, poate defecta placa sau adaptorul USB.

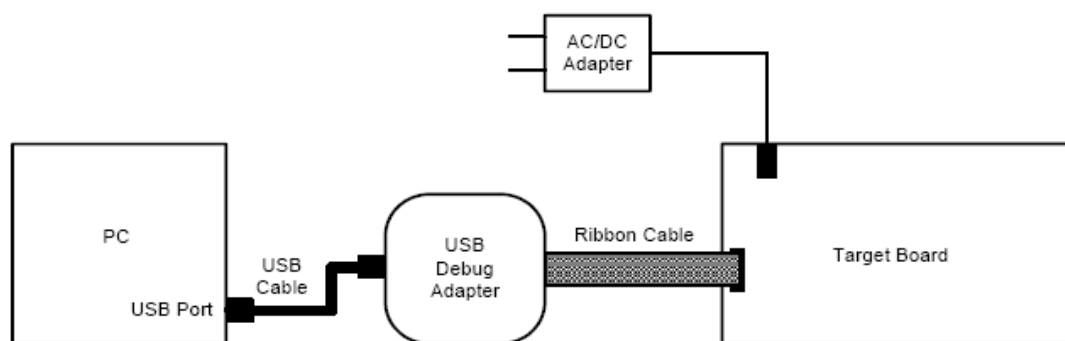


Fig. 2.1.6. Instalare hardware folosind adaptorul USB

2.2 Software pentru dezvoltare: IDE (Integrated Development Environment)

Mediul de dezvoltare contine urmatoarele trei ferestre principale:

- Fereastra Project Viewer
 - File View - folosit pentru vizualizare si management de fisiere asociate proiectului.
 - Symbol View - folosit pentru vizualizarea adreselor simbolurilor.
- Fereastra Edit/Debug
 - Folosita pentru a edita un fisier selectat din proiect.
 - Dupa ce codul a fost incarcat in microcontroler, aceasta fereastra este folosita pentru a observa executia codului in timpul unei depanari.
- Fereastra Output - Este formata din mai multe tab-uri care ofera informatii despre diferite procese in timpul dezvoltarii de aplicatii.
 - Tab-ul **Build** afiseaza rezultatul produs de instrumentele Keil pentru compilare, linkare, asamblare. Se poate da dublu click pe fereastra de Build si fereastra de editare va indica linia unde a aparut eroarea.
 - Tab-ul **List** afiseaza cea mai recenta lista de fisiere generata de asambler sau compilator.
 - Tab-ul **Tool** indica rezultatul unui instrument ales.

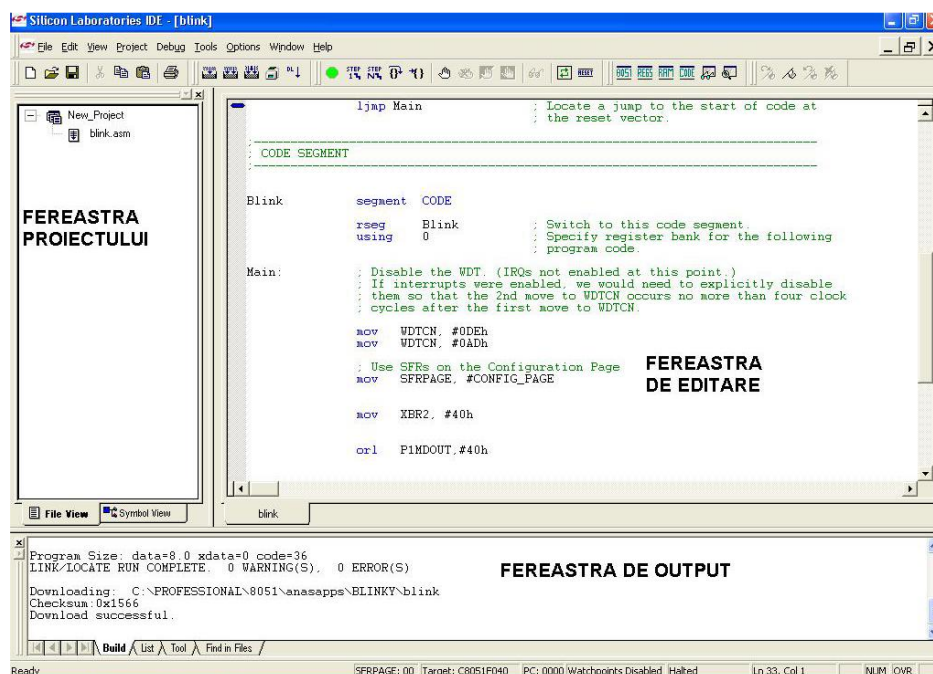


Fig. 2.2. Ferestrele IDE-ului

Executia unui proiect cuprinde urmtorii pasi:

- crearea unui proiect nou: Project -> New Project
- adaugarea unui fisier sursa: Project -> Add Files to Project
- asamblarea fisierului: Project -> Assemble/Compile File
- construirea proiectului: Project -> Build/Make Project
- conectarea la microcontroler: Debug -> Connect
- descarcarea fisierului obiect: Debug -> Download Object File
- executie: Debug -> Go

Observatie:

- conectarea la microcontroler se face dupa ce s-a efectuat instalarea hardware (vezi Lucrarea nr. 1, capitolul 2.1.6)

3 Problema

3.1 Formularea problemei

Sa se implementeze un sistem care sa comande aprinderea periodica a unui led cu o frecventa oarecare.

3.2 Solutie posibila

3.2.1 Descrierea modului hardware

Vom folosi pentru implementarea sistemului numai placa de dezvoltare intrucat putem utiliza led-ul D3 atasat la pinul P1.6 al microcontrolerului.

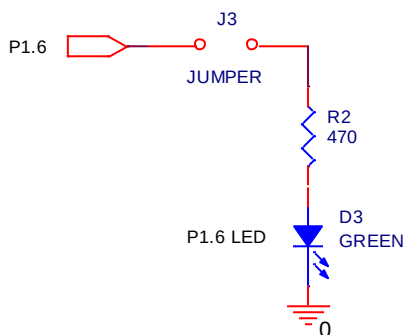


Fig. 3.2.1. Schema de conectare a led-ului D3 la pinul P1.6

3.2.2 Descrierea algoritmului

In general, codul sursa al unui program asm este structurat astfel:

- Zona EQUATES
 - de obicei in aceasta zona vom gasi directiva de includere a fisierului C8051F040.inc care contine etichetele standard pentru microcontrolerul C8051F040
 - in aceasta zona se eticheteaza si adresele altor resurse hardware ale microcontrolerului, pentru a lucra mai usor cu acestea.
- Zona RESET and INTERRUPT VECTORS
 - in aceasta zona se definesc vectorii de intrerupere (daca nu se lucreaza cu intreruperi este necesar sa se defineasca doar vectorul de reset)
 - pentru detalii consultati documentatia asamblorului (fisierul A51.pdf)
- Zona MAIN PROGRAM CODE SEGMENT
 - in aceasta zona se scrie rutina principala a programului
 - de obicei se specifica si segmentul de cod in care se va scrie aceasta rutina
 - pentru detalii despre directivele de asamblare (segment, rseg, cseg) consultati documentatia asamblorului (fisierul A51.pdf)
- Zona FUNCTION CODE
 - prin conventie in aceasta zona se scriu subrutinele programului

In solutia propusa in acest laborator programul va avea in mai multe rutine: main, delay, init.

Rutina principala (main) apeleaza o subrutina de intarziere, apoi schimba starea led-ului. Ultima instructiune a programului reapeleaza rutina principala creand bucla infinita.

Subrutina de intarziere (delay) consta in trei bucle imbricate (loop0, loop1, loop2) avand drept contoare registrele R5, R6, respectiv R7. In cadrul fiecărei bucle apare instructiunea

djnz Rx, eticheta,

cu dublu rol (decrementeaza registrul Rx si face un salt la eticheta specificata in cazul in care valoarea stocata in Rx este nenula). In cazul initializarilor facute in program (R5 = 0, R6 = 0, R7 = 20) bucla exterioara se executa de doua ori, iar buclele interioare de cate 255 de ori.

Subrutina de initializare

- dezactiveaza intreruperile

- in aceasta aplicatie nu se vor folosi intreruperi, prin urmare acestea se dezactiveaza. Mai mult, este recomandat ca dezactivarea watchdog timer-ului sa se faca in timp ce intreruperile sunt inactive.
- dezactivarea intreruperilor se face resetand bitul EA.
- dezactiveaza watchdog timer-ul
 - watchdog timer-ul este un mecanism de resetare a microcontrolerului in cazul aparitiei unei disfunctionalitati in rularea programului. Resetarea apare la un anumit interval de timp (presetate) daca programul nu restarteaza periodic watchdog timer-ul.
 - in aceasta aplicatie nu avem nevoie de un astfel de mecanism de protectie. In consecinta acesta se dezactiveaza
 - dezactivarea watchdog timer-ului se face scriind succesiv in registrul WDTCN valorile 0xDE si 0xAD.
 - pentru detalii despre configurarea watchdog timer-ului si definitia registrului WDTCN consultati documentatia microcontroler-ului (fisierul C8051F04xRev1_4.pdf, pag. 169, pag. 171).
- activeaza crossbar-ul
 - crossbar-ul este un comutator digital care permite maparea resurselor interne ale sistemului la pinii porturilor P0, P1, P2 si P3. Numaratoarele/cronometrele, magistralele seriale, intreruperile hardware, iesirile comparatoarelor si alte semnale digitale din controler pot fi configurate sa apara la pinii porturilor de intrare/iesire, specificati in registrele pentru controlul crossbar-ului.
 - in aceasta aplicatie nu avem nevoie de resurse interne, ci doar de portul P1, deci este nevoie doar de activarea crossbar-ului. Activarea crossbar-ului se face setand bitul 6 al registrului XBR2
 - pentru detalii despre configurarea crossbar-ului si definitia registrului XBR2 consultati documentatia microcontroler-ului (fisierul C8051F04xRev1_4.pdf, pag. 206, pag. 216).
- configureaza portul P1
 - aplicatia foloseste pinul 6 al portului P1 pentru a comanda un led. Deci, pinul P1.6 trebuie setat ca pin digital de iesire. Modul de iesire va fi Push-Pull. Configurarea modului de iesire al portului se face modificand valoarea stocata in registrul P1MDOUT
 - definitia registrului P1MDOUT se afla in documentatia microcontroler-ului (fisierul C8051F04xRev1_4.pdf, pag. 219).

Organigrama programului este prezentata in Fig. 3.2.2.

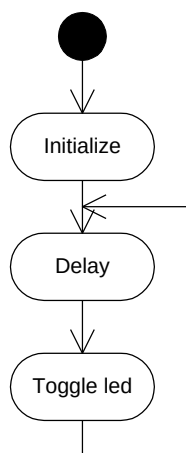


Fig. 3.2.2. Organigrama aplicatiei Blinky

4 Desfasurarea lucrarii

1. Creati un nou proiect folosind fisierul Z:\Lab8051\Laborator1\blinky.asm urmand pasii prezentati la capitolul 2.2. Observatie: Fisierul C8051F040.inc trebuie sa se afle in acelasi director cu proiectul.
2. Executati aplicatia.
3. Modificati programul astfel incat frecventa cu care se aprinde led-ul sa fie mai mica.
4. Calculati frecventa cu care se aprinde led-ul stiind ca procesorul ruleaza la 3MHz si folosind Tabelul 2.1.4.1.4. pentru a determina cati cicli dureaza fiecare instructiune.
5. Modificati frecventa de aprindere a led-ului astfel incat stingerea led-ului sa fie aproape insesizabila. Indicatie: frecventa critica a sistemului vizual uman este aproximativ 25 – 30 Hz.
6. Modificati programul astfel incat led-ul sa se aprinda cu o frecventa crescatoare.
7. Modificati programul astfel incat led-ul sa stea mai mult aprins decat stins. Indicatie: frecventa de aprindere a led-ului ar trebui sa fie relativ mica pentru a putea sesiza palparea, iar semnalul de comanda al led-ului ar trebui sa aiba forma din Fig. 4.7.



Fig. 4.7.

8. Modificati programul astfel incat intr-o perioada led-ul sa clipeasca de trei ori, apoi sa stea stins un interval de timp mai mare. Indicatie: semnalul de comanda al led-ului ar trebui sa aiba forma din Fig. 4.8.



Fig. 4.8.

9. Modificati programul astfel incat apasarea switch-ului S2 de pe placa de dezvoltare sa determine aprinderea led-ului. Schema conectarii switch-ului la microcontroler este prezentata in Fig. 4.9.

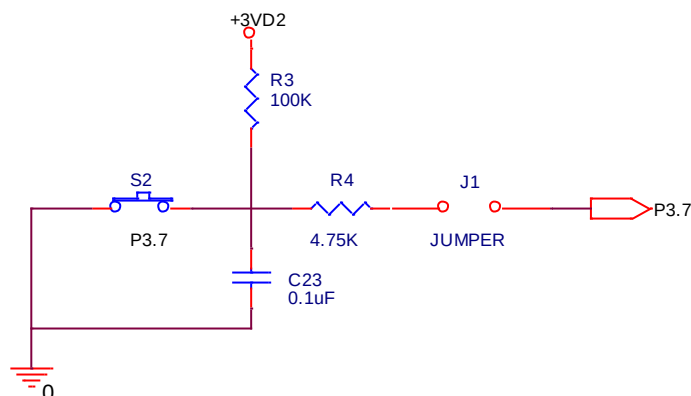


Fig. 4.9. Conectarea switch-ului S2 la pinul P3.7 al microcontrolerului

5 ANEXE

5.1 Codul aplicatiei Blinky

```
;-----  
;  
;  
;  
; FILE NAME : blinky.asm  
; TARGET MCU : C8051F040  
; DESCRIPTION : LED blinking.  
;  
; NOTES:  
;  
;-----  
; EQUATES  
;-----  
  
    $include (c8051f040.inc)                ; Include register definition file.  
    GREEN      equ    P1.6                  ; Label port P1.6 as GREEN (green led)  
  
;-----  
; RESET and INTERRUPT VECTORS  
;-----  
  
                cseg      AT 0x0000          ; Reset Vector  
                ljmp      main              ; Locate a jump to the start of code  
                                                ; at the reset vector.  
;-----  
; MAIN PROGRAM CODE SEGMENT  
;-----  
  
mainCodeSeg      segment      CODE  
                rseg      mainCodeSeg      ; Switch to this code segment.  
                using     0                ; Specify register bank for the  
                                                ; following program code.  
  
main:  
    mainLoop:    acall     init  
                mov       A, #0x02  
                call      delay  
                cpl       GREEN  
                jmp       mainLoop  
  
;-----  
; FUNCTION CODE  
;-----  
  
delay:           mov       R7, A  
    loop1:       mov       R6, #0x00  
    loop0:       mov       R5, #0x00  
                djnz      R5, $  
                djnz      R6, loop0  
                djnz      R7, loop1  
                ret  
  
init:  
                clr       EA                ; Disable global interrupts  
                mov       WDTCN, #0xDE      ; Disable Watch Dog Timer  
                mov       WDTCN, #0xAD  
                mov       SFRPAGE, #CONFIG_PAGE ; Use SFRs in the  
                                                ; configuration Page  
initIOandCross: mov       XBR2, #0x40      ; Enable Crossbar  
                orl       P1MDOUT, #0x40    ; Set P1.6 (GREEN) as digital  
                                                ; output in push-pull mode.  
                clr       GREEN            ; Turn off green led  
  
                ret  
;-----  
; End of file.  
  
END
```